Manual Tecnico

Simulador de EXT2

Guia para el desarrollador (Frontend y Backend)

Proyecto 1
Manejo e Implementación de Archivos
Gisela Mishell Monroy Ovalle
202404409

ÍNDICE

Introducción..... 4
Especificación técnica.....5
Requisitos de hardware.....5
Requisitos de Software..... 5
Arquitectura general..... 5
Flujo del sistema..... 6
Interfaz gráfica.....6
App.jsx.....6
api.js (puente entre React y C++)..... 6
main.jsx, index.htm, App.cpp y package.json..... 7
Archivo .smia.....7
API REST y comunicación Frontend/Backend.....7
POST / api / procesar paso a paso..... 8
http_server.cpp.....8
json_util.cpp..... 8
MAKEFILE.....8
Variables y módulos comunes.....8
Backend..... 10
estado.h (permanece en RAM)..... 10
structs.h (estructuras binarias)..... 11
Analizador (léxico y sintáctico)..... 11
Tokenización.....11
Despacho.....11
Archivo .smia..... 11
Simulación EXT2: cómo se representa en el archivo binario .mia.....12
MKDISK: creación física del disco..... 12
MKFS y distribución física de EXT2.....12
Cálculo utilizado por MKFS.....13
Campos de ubicación del superbloque..... 13
Inodos, bloques y bitmaps.....13
Inodo..... 13
Bloque de carpeta..... 13
Bloque de archivo.....14
Asignación.....14
Usuarios, grupos, archivos y permisos..... 14
MKFS crea users.txt.....14
LOGIN..... 14
MKGRP / RMGRP / MKUSR / RMUSR / CHGRP.....14
MKDIR y MKFILE..... 15
Permisos..... 15
Reportes REP..... 15

Introducción

Este es un simulador de un sistema de archivos de tipo ext2, se simula la creación de discos, de particiones, el montaje de estas particiones, inicio de sesión al sistema, generación de reportes, etc. Toda la información es manejada por medio de la creación de archivos binarios con extensión .mia, de tamaño fijo, aquí se simula la creación de discos, el montado, formateado, eliminado, etc.

Objetivos:

Brindar una idea clara de cómo fue creado el programa, para que cualquier otra persona pueda interpretar de una buena manera el código creado, incluso es viable ver este manual para cuando se le desea hacer una actualización el programa, se sabe dónde se debe de hacer, debido a que en este documento se explica de forma clara y ordenada las funciones del código, también busca Explicar qué hace cada capa, qué archivo es responsable de cada tarea, cómo viaja un comando desde la interfaz hasta C++, cómo funciona el Makefile y cómo una partición EXT2 se representa físicamente dentro de un .mia.

Dirigido a:

Está dirigido a todas aquellas personas que deseen modificar o actualizar el programa, también para quienes deseen un programa como este, para que sepan cómo deben leer el código y como es su funcionamiento, incluso está dirigido a el programador en sí, ya que de esta forma puede saber de una manera más clara y ordenada que es y dónde está el código que desea implementar o actualizar.

Especificación técnica

Requisitos de hardware

- Disco duro capaz de correr el programa, a partir de 250 GB
- Procesador Core i5 o superior
- 4GB de memoria RAM como mínimo.

Requisitos de Software

- Debe tener instalado algún editor de código fuente, para usar el programa directamente, Computadora de escritorio o portátil, se recomienda utilizar Visual Studio Code.
- Instalar previamente las dependencias necesarias, específicamente:
    - C++ (lenguaje de programación con el que se desarrolló todo)
    - npm para correr el servidor
    - make (para la api)

Arquitectura general

El proyecto está dividido en 4 capas: interfaz web, API/servidor HTTP, núcleo del backend y almacenamiento persistente. El frontend envía una petición HTTP a la API. La API entrega el texto al analizador, el analizador válida y despacha el comando, y el comando modifica el estado correspondiente, incluyendo el archivo .mia cuando la operación es persistente.

Capa	Responsabilidades	Archivos principales
Frontend	Captura comandos y archivos .smia y muestra los resultados	Frontend/src/App.jsx, api.js, main.jsx, App.css
API/HTTP	Recibe HTTP, extrae JSON, llama al analizador y devuelve un JSON	api_server_main.cpp, http_server.cpp, json_util.cpp
Analizador	Tokeniza, válida léxicamente y sintácticamente, despacha	analizador.cpp, validaciones.cpp
Comandos	Implementan la semántica de MKDISK, FDISK, MOUNT y MKFS	mkdisk.cpp, fdisk.cpp, mount.cpp, mkfs.cpp…
EXT2 / Persistencia	Lee/escribe superbloque, bitmaps, inodos y bloques en .mia	ext2_utils.cpp, disk_utils.cpp, structs.h

Build	Compila consola y API con los mismos módulos comunes	Makefile

Flujo del sistema

1. **React**: app.jsx mantiene el texto y llama a api.js
2. **HTTP**: fetch haec POST/api/procesar a localhost: 8080
3. **API C++**: api_server_main.cpp recibe JSON y llama a procesar_archivo_comandos().
4. **Analizador**: por medio de lexical_analysis valida las comillas, valida el lexico, luego la sintaxis y por último despacha, con dispatcher.
5. **Comando**: Ejecuta semántica y si corresponde lee/escribe en el .mia
6. **Respuesta**: el resultado textual vuelve como JSON.
7. **React**: App.jsx coloca el reporte en el textarea de Resultados.

Interfaz gráfica

Se utiliza React + Vite. La interfaz actual está construida con React y Vite. React administra el estado de la pantalla y Vite proporciona el entorno de desarrollo/build. El Frontend no contiene la lógica de EXT2: su función principal es interacción con el usuario y comunicación con la API.

App.jsx

- comandos: contiene el texto que el usuario escribió o el contenido leído desde un archivo .smia.
- resultados: contiene la respuesta textual devuelta por el backend.
- procesando: evita ejecutar otra petición mientras una está en curso y cambia el botón a “Procesando...” .
- archivo Seleccionado: guarda el nombre del .smia que se cargó.
- El botón Subir .smia usa FileReader para leer el archivo como texto. No trata un .smia como disco.
- El botón Limpiar borra comandos, resultados y nombre del archivo.
- El botón Procesar llama a procesarArchivo(comandos).

api.js (puente entre React y C++)

Este archivo concentra la comunicación de red. La aplicación usa fetch nativo del navegador; no necesita una biblioteca
HTTP adicional.



```
const URL_BASE = 'http://localhost:8080'

export async function procesarArchivo(contenido) {
  const respuesta = await fetch(`${URL_BASE}/api/procesar`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ contenido }),
  })

  const datos = await respuesta.json()

  if (!respuesta.ok) {
    throw new Error(datos.error || 'Error desconocido del backend')
  }

  return datos.reporte
}
```

main.jsx, index.htm, App.cpp y package.json

- main.jsx monta el componente App dentro de #root y carga App.css.
- index.html es el documento base que Vite sirve al navegador.
- App.css contiene la presentación visual de la interfaz.
- package.json declara React, ReactDOM y Vite; los scripts principales son dev, build y preview.

Archivo .smia

El archivo .smia es un archivo de comandos. React lo lee con FileReader y copia su contenido al textarea. Después, el usuario presiona Procesar y ese texto se manda a /api/procesar. El .smia no se envía como un disco binario ni se convierte en .mia.

API REST y comunicación Frontend/Backend

El backend ejecutable api_server levanta un servidor HTTP propio en el puerto 8080. La implementación está hecha en C++ usando sockets POSIX; no depende de un framework HTTP externo. api_server_main.cpp registra las rutas y conecta cada ruta con una función manejadora

POST / api / procesar paso a paso

El frontend se queda escuchando el puerto 8080, se manda por medio de un JSON el comando o contenido hacia el backend, ahi el backend extrae cada campo del JSON, procesa el archivo, lo ejecuta y manda la salida por un JSON al frontend nuevamente

http_server.cpp

- Abre un socket y escucha en 8080.
- Recibe la petición HTTP y separa método, ruta y cuerpo.
- Busca la ruta registrada y ejecuta el callback correspondiente.
- Construye la respuesta HTTP y la devuelve al navegador.
- ncluye cabeceras CORS y atiende OPTIONS para permitir la comunicación entre el puerto del Frontend y el 8080.

json_util.cpp

El proyecto no utiliza una biblioteca JSON externa para estas peticiones simples. extraer_campo_json() obtiene campos de entrada y escapar_json() prepara texto para incorporarlo correctamente al JSON de salida.

La comunicación, por tanto, no es “React llama directamente a mkdisk.cpp”. La cadena correcta es: **React → fetch → HTTP server → API handler → analizador → comando → archivo .mia/RAM → respuesta → React**.

MAKEFILE

El archivo se llama Makefile. Es un archivo de reglas para automatizar la compilación. En lugar de escribir manualmente un comando g++ enorme cada vez, se ejecuta make y el sistema utiliza las reglas declaradas

Variables y módulos comunes

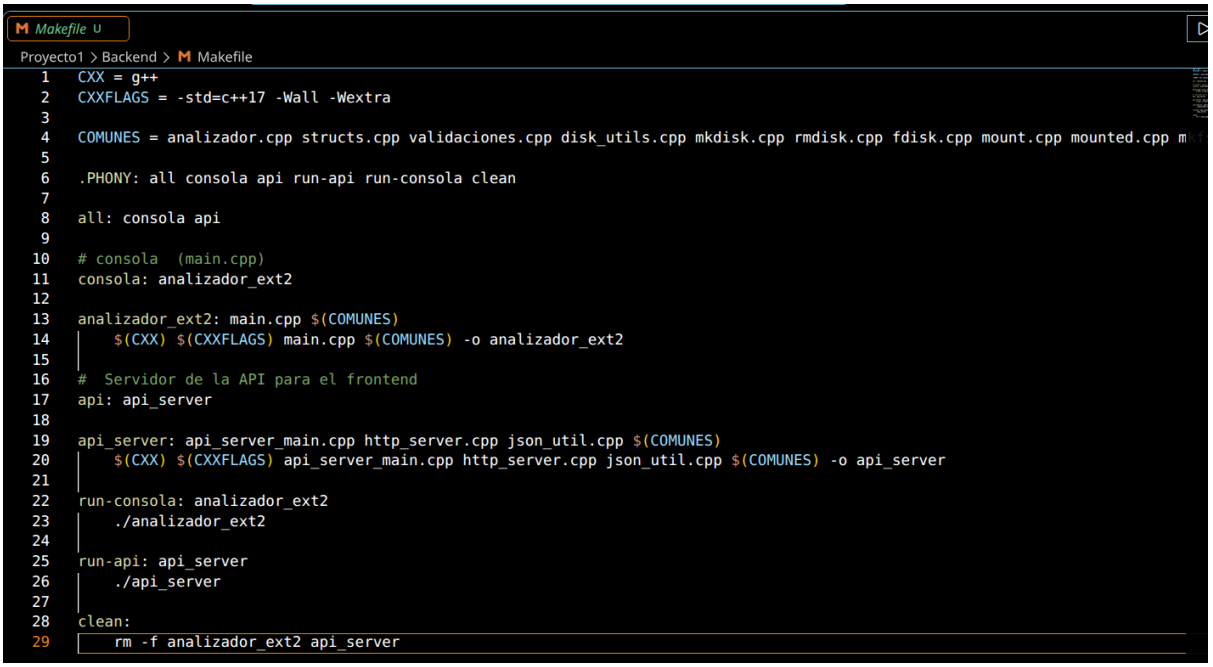
CXX = g++
CXXFLAGS = -std=c++17 -Wall -Wextra
COMUNES = analizador.cpp structs.cpp validaciones.cpp disk_utils.cpp …

COMUNES contiene los módulos que comparten los dos ejecutables: el modo consola y el servidor de la API. Esto evita
mantener dos copias de la lógica.

comandos del Makefile

Comando	Responsabilidad
make	compila la api_server
make run-consola	compilar el ejecutable de consola y lo ejecuta
make run-api	compila api_server y deja el servidor escuchando 8080
make clean	elimina los binarios compilados

Si se modifica analizador.cpp, ext2_utils.cpp, mkfile.cpp, reports.cpp o cualquier otro archivo incluido en COMUNES, la regla recompila esos archivos junto con el punto de entrada correspondiente. Para el proyecto es especialmente útil porque consola y API deben utilizar exactamente la misma lógica de comandos.



Backend

El Backend está organizado por responsabilidad. Los archivos .h declaran estructuras y funciones; los .cpp implementan la lógica. La parte central puede entenderse en seis grupos.

Grupo	Responsabilidad	Archivos principales
Entrada	Modo consola	main.cpp
Analizador	Tokenización, procesamiento de líneas y despacho	analizador.cpp y .h
Validación	validación léxica, sintáctica y comillas	validaciones.cpp y .h
Estado / estructuras	estado compartido en RAM y estructuras binarias	estado.h, structs.cpp y .h
Disco	lectura/escritura de MBR, EBR y búsqueda de particiones.	disk_utils.cpp y .h
EXT2	acceso al superbloque, bitmaps, inodos, bloques, rutas, usuarios y permisos	ext2_utils.cpp y .h
Comandos	comandos a ejecutar sobre el disco	mkdisk, rmdisk, fdisk, mount, mounted, mkfs, login, logout, mkgro, etc.
Reportes	REP: mbr, disk, inode, block, bitmaps, tree, sb, file y ls	reports.cpp y .h
HTTP / API	exposición del backend mediante HTTP / JSON	api_server_main.cpp, http_server.cpp y .h, json:util.cpp y .h

estado.h (permanece en RAM)

- discos: cache del MBR de discos conocidos; la fuente persistente real es el .mia.
- discosMontados: MOUNT es deliberadamente RAM-only.

- particionesFormateadas: cache del estado de formato.
- Las estructuras antiguas usuarios, archivos y carpetas permanecen por compatibilidad con la fase anterior; ya no son la
- fuente principal del contenido persistente.
- sesion: usuario, id, uid y gid de la sesión actual; permanece en RAM

structs.h (estructuras binarias)

Las estructuras que se escriben directamente en .mia no contienen std::string ni std::vector. Esto es importante porque el archivo binario necesita una representación de tamaño fijo y directamente serializable.

Analizador (léxico y sintáctico)

El analizador utiliza validarLexico() para verificar que comando y nombres de parámetros tengan una forma léxica válida, validarSintaxis() y validarComillasCerradas() sirve para verificar que los parámetros tengan una estructura válida y las comillas están cerradas.

Tokenización

lexical_analysis() recorre la línea carácter por carácter. Identifica el comando, parámetros que comienzan con '-', valores y cadenas entre comillas. Por ejemplo, una ruta como -path="/tmp/Mi Disco.mia" puede conservar el espacio interno.

Despacho

despacharComando() convierte el nombre del comando a minúsculas y selecciona la función correspondiente: mkdisk(), fdisk(), mount(), mkfs(), login(), mkfile(), mkdir(), cat(), rep(), etc.

Archivo .smia

procesar_archivo_comandos() separa el contenido en líneas, conserva el número original de línea, ignora líneas vacías y comentarios que empiezan con # y procesa cada línea independientemente. Un error en una línea no obliga a detener el procesamiento de las siguientes.

Simulación EXT2: cómo se representa en el archivo binario .mia

Esta es la diferencia fundamental respecto de la primera versión del proyecto. El archivo .mia es ahora el almacenamiento persistente de la simulación. El programa usa operaciones binarias seekg/seekp/read/write para acceder a posiciones concretas del disco simulado.

MKDISK: creación física del disco

MKDISK valida tamaño, unidad, fit, ruta y extensión .mia. Luego crea las carpetas padre si son necesarias, crea el archivo, lo llena con ceros hasta alcanzar el tamaño solicitado y escribe un MBR al comienzo

- MBR: El MBR se escribe al offset 0 del .mia. Contiene tamaño total, fecha, firma, fit y las cuatro entradas de partición. disk_utils.cpp ofrece leerMBR() y escribirMBR().
- FDISK: primero lee el MBR del .mia, calcula espacios disponibles y aplica first fit, best fit o worst fit según el fit correspondiente. Cuando encuentra una ubicación, actualiza la entrada de partición y vuelve a escribir el MBR. En una extendida también puede escribir EBRs físicamente dentro de la región extendida.
- MOUNT: relaciona una partición existente del .mia con un ID de montaje almacenado en RAM. En esta versión el montaje no se persiste dentro del archivo; al reiniciar el programa se debe volver a montar. El ID utiliza la convención configurada para el proyecto, con base de carnet 09.

MKFS y distribución física de EXT2

MKFS recibe un ID montado, obtiene el inicio y tamaño de la partición y calcula cuántos inodos y bloques caben. Después escribe la estructura EXT2 directamente dentro de la región de esa partición.

Orden	Región	Contiene
1	Superbloque	Información global: cantidad de inodos/bloques, libres, tamaños, magic, offsets.
2	Bitmap de inodos	Un byte por inodo; 0 libre, 1 ocupado.
3	Bitmap de bloques	Un byte por bloque; 0 libre, 1 ocupado.
4	Tabla de inodos	Array de estructuras inodo de tamaño fijo.

5	Bloques	Bloques de carpetas, archivos o apuntadores.

Cálculo utilizado por MKFS

sb = sizeof(sbloque)
ino = sizeof(inodo)
bc = sizeof(bcarpetas)
n = (size - sb) / (1 + 3 + ino + 3*bc)
blocks = 3*n

El código actual calcula el número de inodos con esa expresión y utiliza tres veces esa cantidad como número de bloques. Luego determina offsets consecutivos para bitmap de inodos, bitmap de bloques, tabla de inodos y bloques.

Campos de ubicación del superbloque

s_bm_inode_start = start + sizeof(sbloque)
s_bm_block_start = s_bm_inode_start + n
s_inode_start = s_bm_block_start + blocks
s_block_start = s_inode_start + n*sizeof(inodo)

Estos offsets permiten que las funciones de EXT2 encuentren una estructura con una operación del tipo: offset = s_block_start + índice * tamaño_del_bloque.

Inodos, bloques y bitmaps

El sistema utiliza estructuras binarias de tamaño fijo. Los índices de los bitmaps identifican inodos y bloques físicos dentro
de la partición EXT2.

Inodo

inodo contiene UID, GID, tamaño, tiempos de acceso/creación/modificación, 15 referencias de bloque, tipo y permisos.
i_type=0 representa carpeta e i_type=1 archivo.

Bloque de carpeta

bcarpetas tiene cuatro entradas content. Cada entrada almacena nombre y número de inodo. MKFS crea la raíz con '.' y '..' y agrega users.txt como entrada inicial.

Bloque de archivo

barchivos contiene 64 bytes de contenido. Cuando un archivo necesita más bloques, ext2_utils calcula el índice lógico y utiliza referencias directas, indirectas simples y dobles; la lectura contempla también la estructura de triple indirección.

Asignación

- asignarInodo() busca el primer byte 0 del bitmap de inodos, lo cambia a 1 y actualiza contadores del superbloque.
- asignarBloque() hace lo mismo con el bitmap de bloques.
- liberarInodo() y liberarBloque() vuelven a marcar el elemento como libre y actualizan el superbloque.

Usuarios, grupos, archivos y permisos

La información persistente de usuarios y grupos está dentro de /users.txt en la partición EXT2. La sesión actual sí se mantiene en RAM.

MKFS crea users.txt

1, G, root
1, U, root, root, 123

MKFS crea el inodo de la raíz, el inodo de users.txt, sus bloques y las entradas correspondientes en la carpeta raíz.

LOGIN

LOGIN busca el ID montado, abre la partición EXT2, localiza /users.txt, lee su contenido y verifica usuario, grupo y contraseña. Si es correcto, guarda en estado.sesion el usuario, ID, UID y GID.

MKGRP / RMGRP / MKUSR / RMUSR / CHGRP

Estos comandos requieren sesión activa y actualizan físicamente /users.txt. Las operaciones de eliminación marcan los
registros como eliminados mediante el mecanismo implementado en el archivo de usuarios.

MKDIR y MKFILE

- MKDIR asigna un inodo y un bloque de carpeta, crea '.' y '..' y agrega la entrada en el directorio padre.
- MKFILE asigna un inodo y tantos bloques de archivo como sean necesarios. -cont tiene prioridad sobre -size; con -size se
- genera la secuencia 0123456789... .
- CAT lee inodos/bloques del .mia y valida permisos de lectura

Permisos

ext2_utils.cpp interpreta permisos UGO a partir de i_perm y considera al usuario root con UID 1 como privilegiado. El permiso efectivo se selecciona según si el UID coincide con el propietario, si el GID coincide con el grupo o, en otro caso, usando los bits de otros.

Reportes REP

reports.cpp lee el .mia y genera los reportes solicitados. Los reportes gráficos se construyen como DOT y se renderizan con Graphviz; los bitmaps y file producen texto directamente.

Reporte	Fuente principal	Contenido
mbr	MBR + EBR	Información del disco, MBR y EBRs.
disk	MBR	Distribución del disco y porcentajes.
inode	Bitmap + inodos	Solo inodos utilizados.
block	Bitmap + inodos + bloques	Solo bloques utilizados.
bm_inode	Bitmap de inodos	Todos los bits, 20 por línea.
bm_block / bm_bloc	Bitmap de bloques	Todos los bits, 20 por línea.
tree	Inodos + bloques	Árbol EXT2 completo alcanzable.
sb	Superbloque	Campos del superbloque.
file	Inodo + bloques	Contenido de un archivo.
ls	Inodo + directorios	Nombre, tipo, permisos, UID, GID, tamaño y fechas.